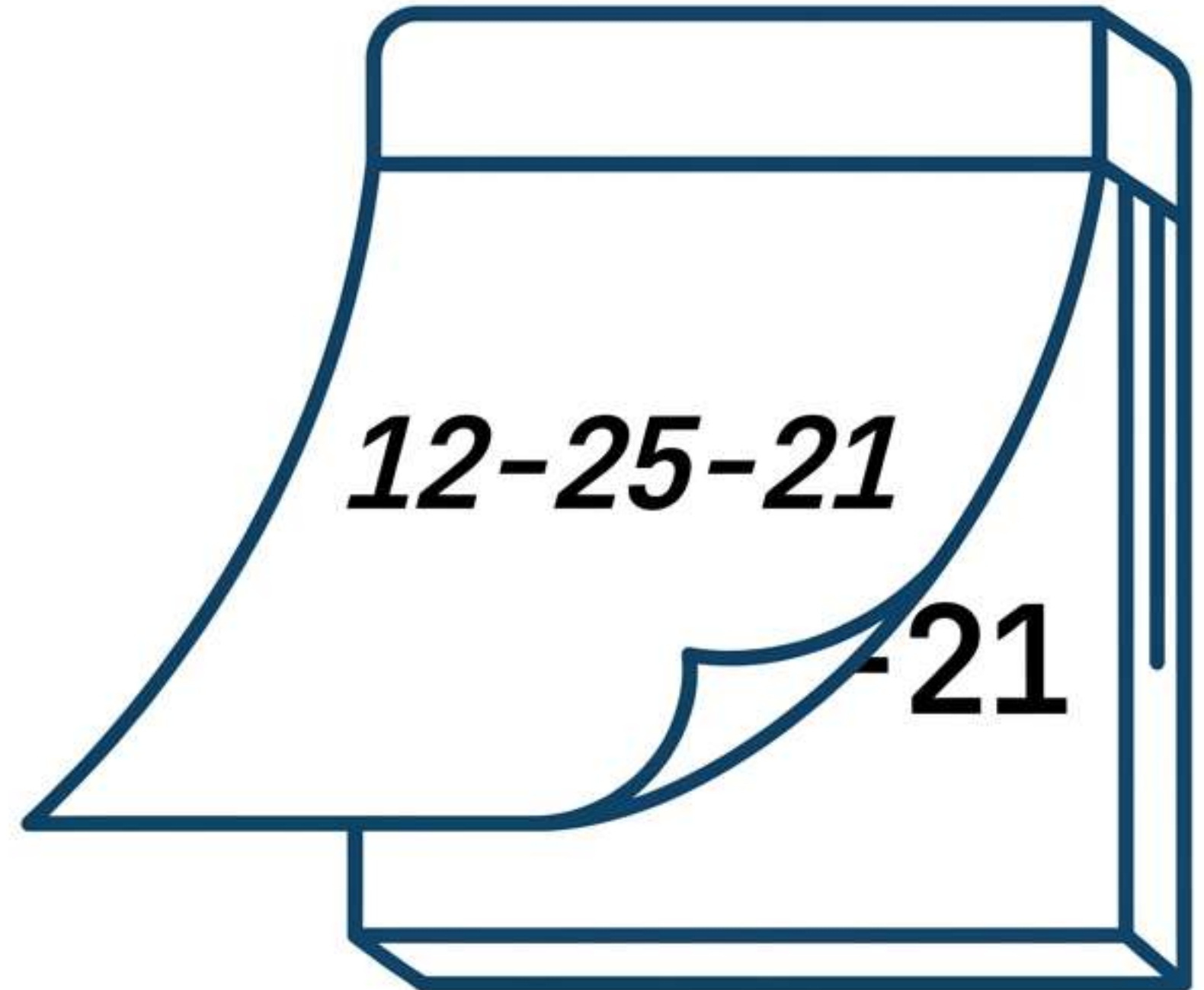


Date Format Conversion

String Manipulation
& Logic Flow



Academic Context: Date String Reversal

The Objective

INPUT (mm-dd-yy)

12-25-21

Function Logic

OUTPUT (yy-dd-mm)

21-25-12

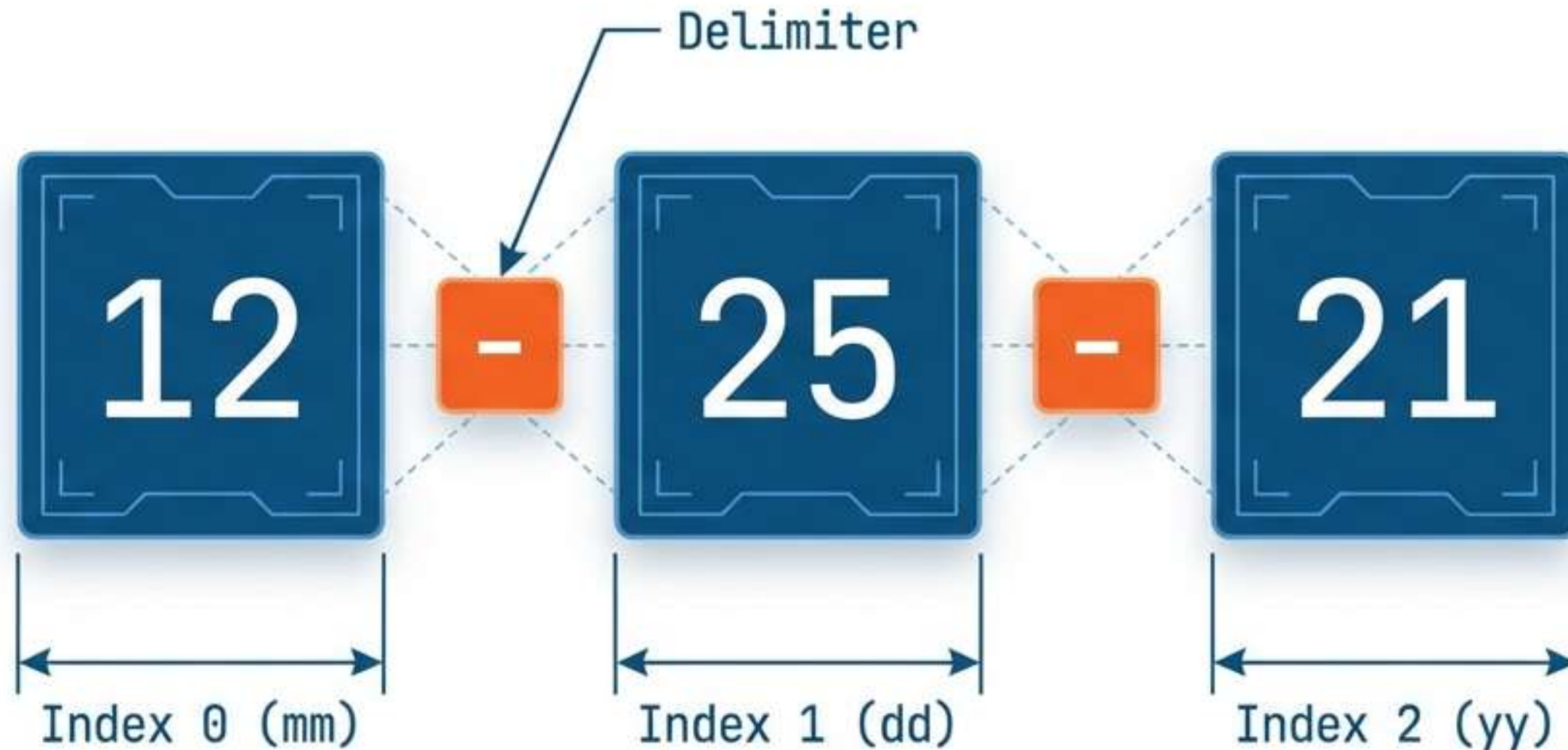
Constraint

No input/output handling required.
Function definition only.

Rule

Rearrangement only. No
modification of component values.

Anatomy of the Input String



Strings are immutable. To manipulate these components, we must first break the structure.

The Algorithmic Strategy



Logic Flow: Split → Reorder → Join

Step 1: Deconstruction

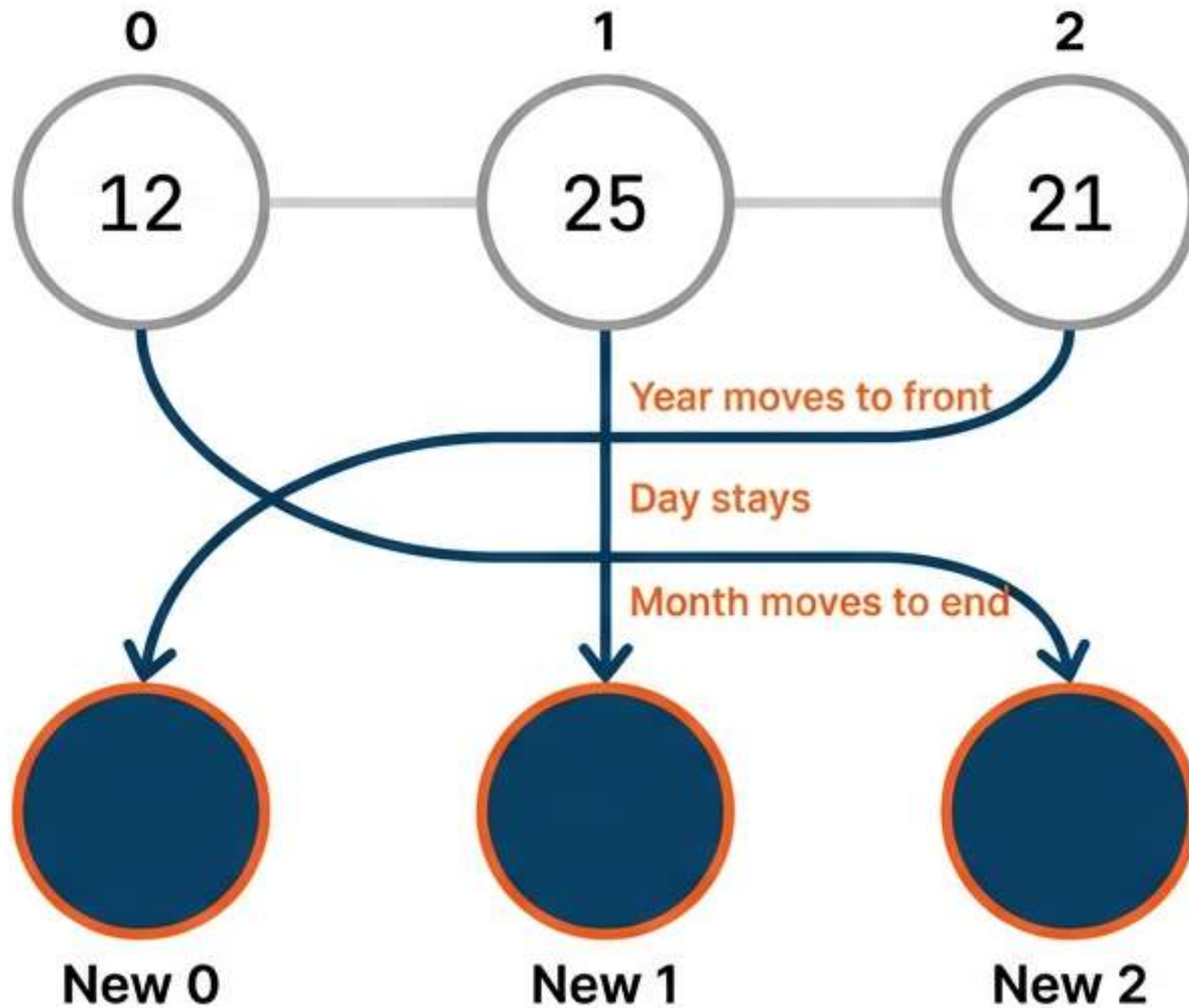
```
parts = date.split('-')
```

Tokenizes string based on hyphen delimiter



[12 , 25 , 21]

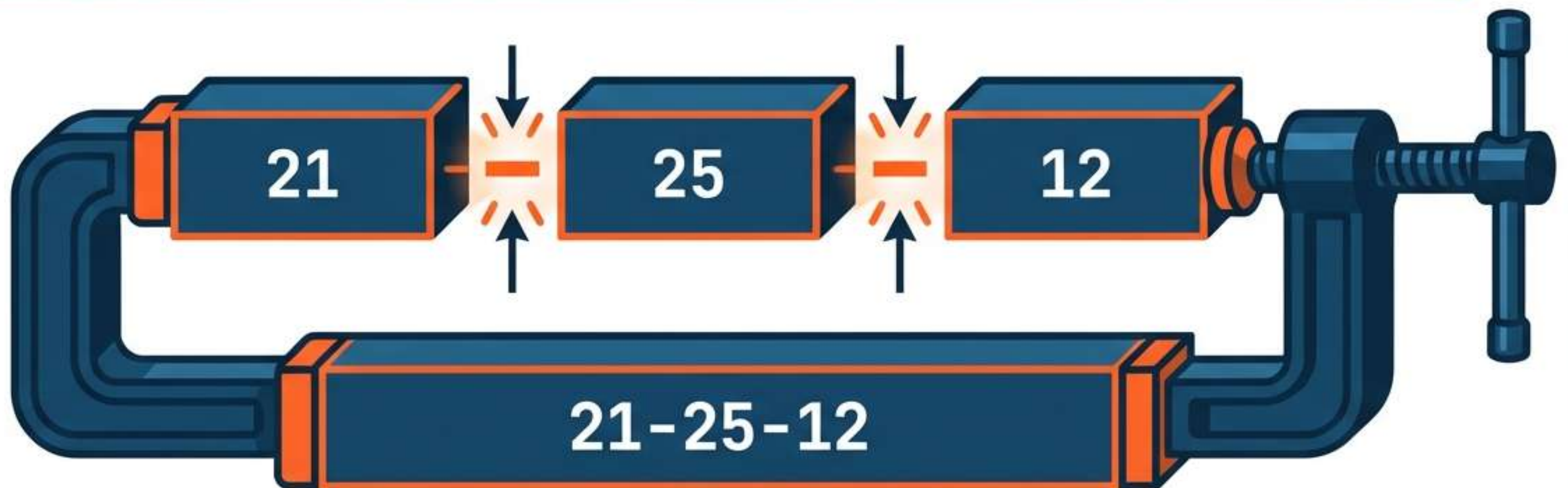
Step 2: The Shuffle



```
parts[2] = '21' (Year)  
parts[1] = '25' (Day)  
parts[0] = '12' (Month)
```


Step 3: Reconstruction

```
return '-'.join([parts[2], parts[1], parts[0]])
```



The `.join()` method creates a new string by concatenating list elements with the specified separator.

Implementation (Standard Approach)

```
def mm_dd_yy_to_yy_dd_mm(date: str) -> str:
```

```
    parts = date.split('-')
```

```
    return '-'.join([parts[2], parts[1], parts[0]])
```

Deconstruct

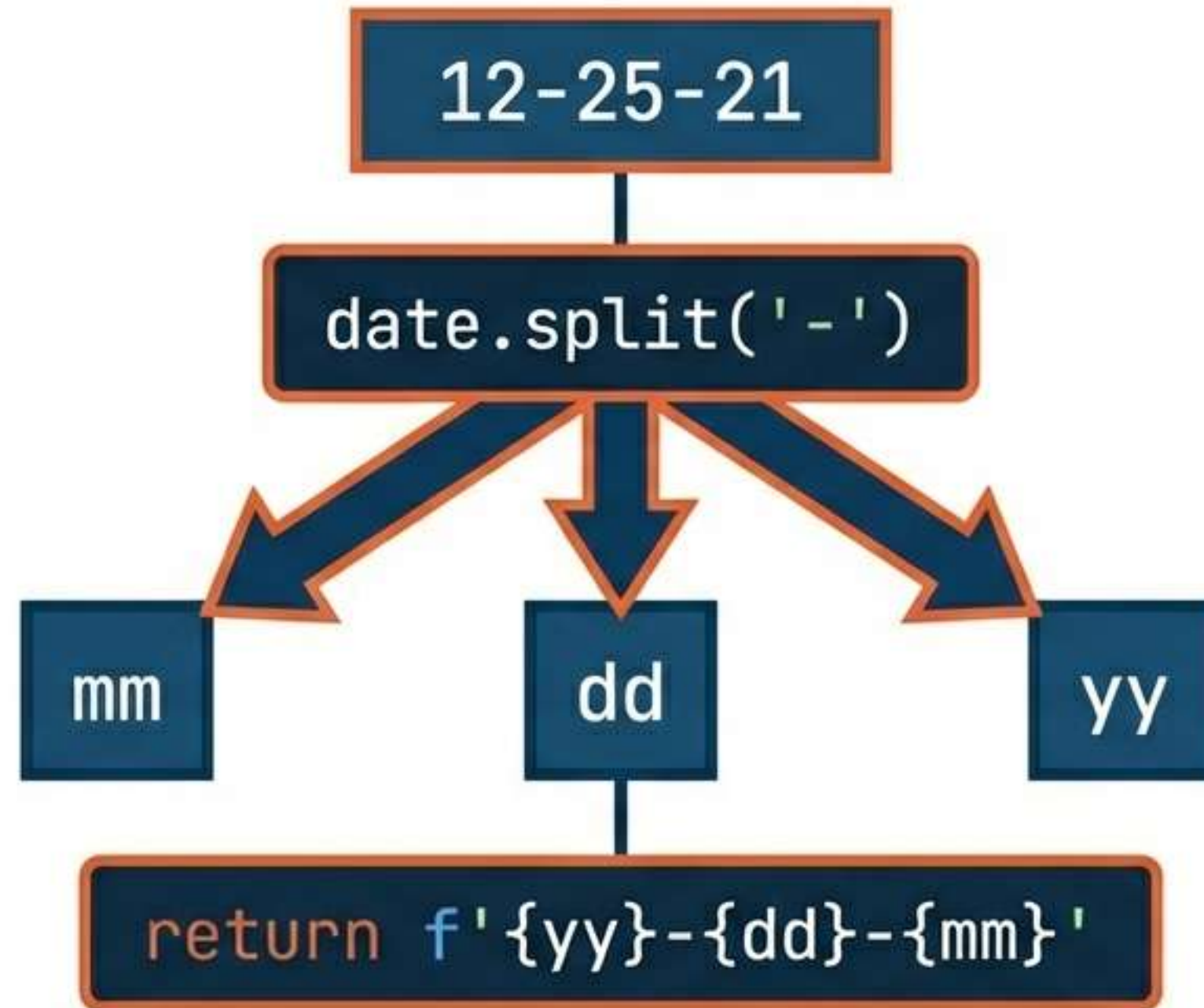
Explicit Reordering

Reconstruct

Pros: Explicit logic. Cons: Verbose indexing.

The 'Pythonic' Approach

Tuple Unpacking



Unpacking assigns semantic meaning immediately, removing the need for raw numbers like `[0]` or `[1]`.

Comparative Analysis

Indexing (Standard)

```
parts[0]  
parts[1]  
parts[2]
```

Explicit Control. Best for arbitrary manipulation.

Unpacking (Pythonic)

```
mm, dd, yy  
= parts
```

Semantic Clarity. Best when structure is fixed and known.

Both valid. Unpacking is preferred for readability in this specific context.

Complexity & Efficiency

Time Complexity

$O(1)$

Constant Time

The input length is fixed to the specific date format (8 characters). It does not scale.

Space Complexity

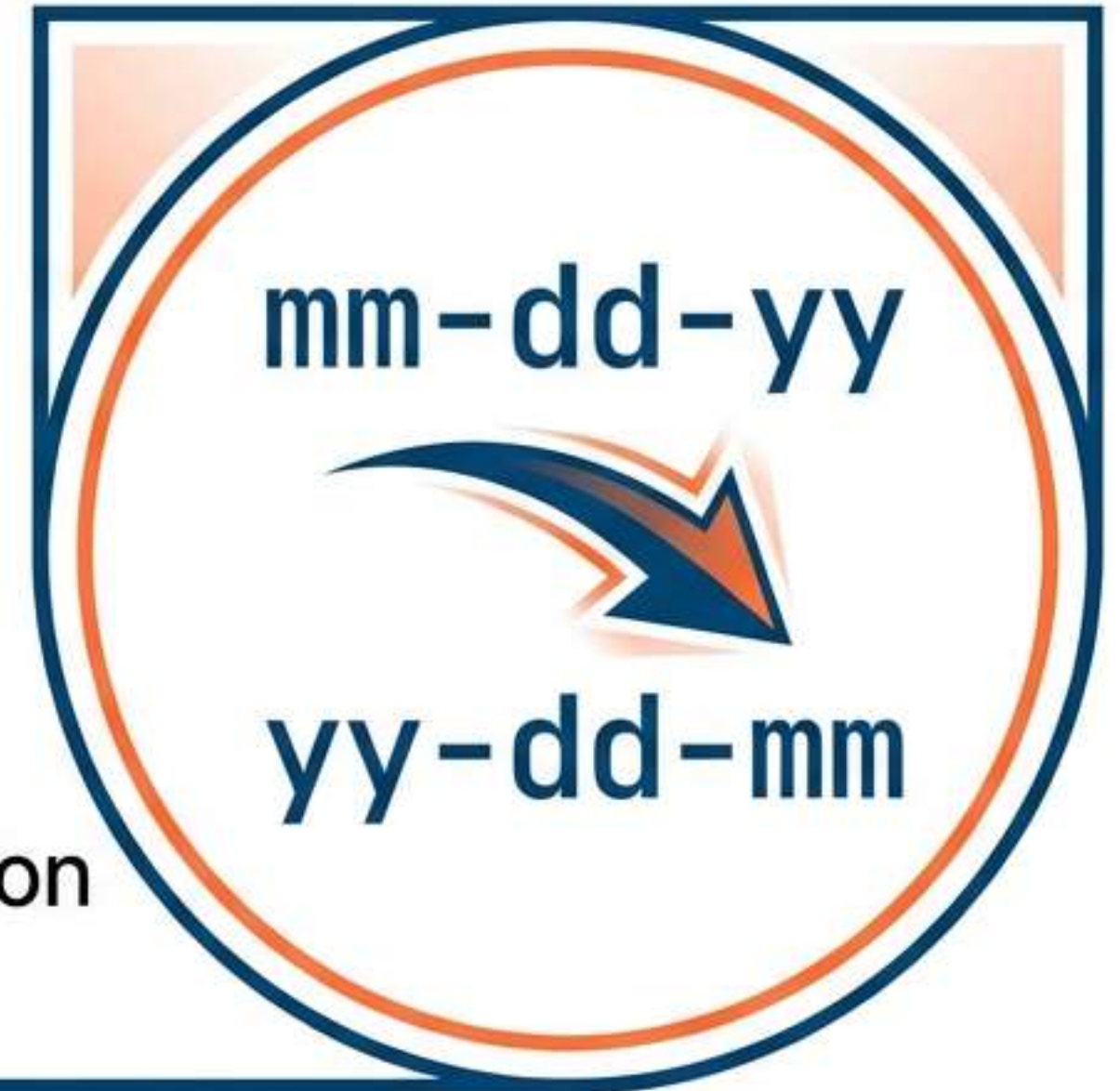
$O(1)$

Constant Space

We only store a fixed number of temporary variables (3 parts), regardless of system load.

Toolbox & Summary

- ✓ **split():** Deconstruct string into list tokens
- ✓ **List Indexing:** Access specific positions
- ✓ **join():** Reconstruct list into string
- ✓ **Tuple Unpacking:** Assign variables by position



Mastering these string manipulation primitives is essential for efficient data formatting.